PROJECT DOCUMENTATION

# Interactive 3D Flipbook Product Catalogue

A responsive, brandable page-flip catalogue website built with React, TypeScript, Vite and Tailwind CSS



*Catalogue cover image used as page 1 of the flipbook (sample placeholder artwork)*

| | |
|---|---|
| Package name | flipbook-product-catalogue (version 1.0.0, private) |
| Type | Static front-end web app (no backend, API keys or database) |
| Stack | React 19, TypeScript, Vite, Tailwind CSS 4, Motion, Lucide icons |
| Catalogue size | 10 pages, 16 images, 21 source components |
| Developer credit in app | Developed by Kazi Jawad (configurable) |

# Contents

Contents . . . . . 2

1. Project Overview . . . . . 3

2. Feature Overview . . . . . 4

3. Technology Stack . . . . . 5

4. Architecture . . . . . 6

5. The Page-Flip Engine . . . . . 7

6. The 10-Page Demo Catalogue . . . . . 8

7. Ready-Made Page Templates . . . . . 13

8. Controls and Interface Reference . . . . . 14

9. Customisation Guide . . . . . 15

10. Build and Deployment . . . . . 16

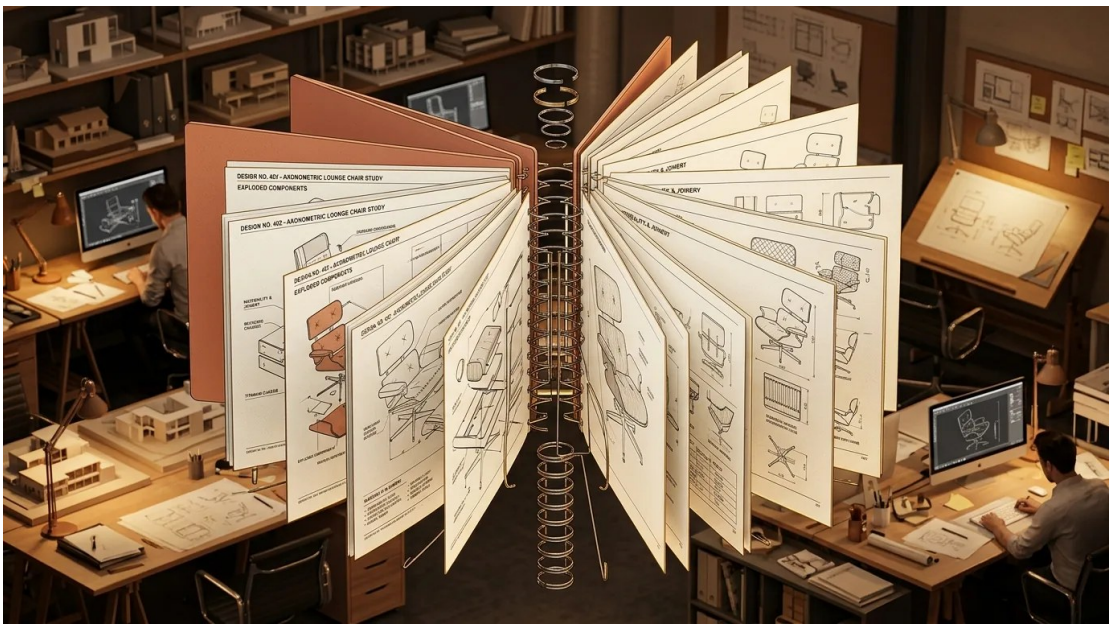
11. Review Notes and Recommendations . . . . . 17

> **How this document was produced.** It is based on a read-through of the project source (components, configuration, README and the Fiverr gig file) and its image assets. The project was **not installed, built or run** for this document, so build size and performance figures are quoted from the project README rather than measured.

# 1. Project Overview

This project is an **interactive 3D flipbook product catalogue**: a website that presents a catalogue as a bound book whose pages physically turn in 3D when a visitor clicks, swipes, drags or uses the keyboard. It is designed as a **white-label template**. A developer replaces the brand details and page images, runs one build command, and publishes the resulting static files on any web host.

The bundled demo is a furniture catalogue (office seating, lounge pods, tables, storage, materials and a BIM toolkit). The furniture content is placeholder material for demonstration; the codebase itself is industry-neutral and is positioned for furniture, real estate, fashion, construction, manufacturing, restaurants, lookbooks and portfolios.



*Concept render used on the "Flipbook Concept" page (page 2)*

## At a glance

| Aspect | Detail |
|---|---|
| Purpose | Turn a static PDF-style catalogue into an engaging, shareable, mobile-friendly web experience |
| Delivery model | Static files in `dist/` after `npm run build`; deployable to Netlify, Vercel, Cloudflare Pages, GitHub Pages or cPanel hosting |
| Customisation | Two config files (`brand.ts`, `pages.ts`) plus an images folder |
| Page types | Plain full-page image, or a custom React component per page |
| Interaction | 3D page turn, drag-to-peel, zoom, fullscreen, autoplay, keyboard, index, thumbnails, share, enquiry, right-click menu, help |
| Source volume | About 4,776 lines of TypeScript, TSX and CSS across the `src` folder |
| Runtime requirement | Node.js 18 or newer for development and build; any static host for deployment |

# 2. Feature Overview

## Experience features

- **Realistic 3D page turn** - each leaf bends as it swings over a spiral binding, with shadows, driven frame-by-frame by `requestAnimationFrame`.
- **Drag-to-peel** - on touch and mouse, dragging a page follows the pointer; releasing past roughly 28% of the drag range completes the turn, otherwise the page springs back.
- **Synthesised page-flip sound** - generated live with the Web Audio API (filtered pink noise); no audio files are shipped, and it can be muted.
- **Zoom** - toggles a zoomed view of the current page (drag-to-turn is disabled while zoomed).
- **Autoplay slideshow** - advances every 4.5 seconds and loops back to page 1 after the last page.
- **Fullscreen mode** - uses the browser Fullscreen API and stays in sync if the user exits with Esc.
- **Fully responsive** - desktop, tablet and phone layouts; the control bar collapses secondary actions into a "More Options" overflow on small screens.

## Navigation and discovery

- **Control bar** - table of contents, all-pages thumbnails, first / previous / next / last, and a "jump to page" number field.
- **Index modal** - a searchable table of contents; the search matches section titles, index titles and "page N".
- **Pages modal** - a responsive thumbnail grid (one to three columns) that updates automatically when pages are added.
- **Custom right-click menu** - replaces the browser menu with catalogue actions (previous, next, cover, back cover, index, pages grid, share, plus view toggles).

## Engagement and sharing

- **Share modal** - copy link, native device share sheet (falls back to copy when unsupported), and email.
- **Concierge (enquiry) modal** - a contact panel showing phone, email, address and opening hours plus an enquiry form (name, email, project scope, message).
- **Closing page** - a branded thank-you page with contact links (email, LinkedIn, GitHub, website) configured in one place.

# 3. Technology Stack

| Technology | Version (package.json) | Role in the project |
|---|---|---|
| React | ^19.0.1 | UI components and state (hooks only; no external state library) |
| TypeScript | ^7.0.2 | Typed components and configuration; `npm run lint` runs `tsc --noEmit` |
| Vite | ^8.3.0 | Dev server (port 3000) and production build |
| @vitejs/plugin-react | ^6.1.1 | React support in Vite |
| Tailwind CSS + @tailwindcss/vite | ^4.3.3 | Utility-first styling; brand accent exposed as CSS variables |
| Motion | ^12.23.24 | Animation library available to the UI |
| lucide-react | ^0.546.0 | Icon set for controls and modals |
| Google Fonts | loaded in index.html | Playfair Display (headings) and Plus Jakarta Sans (UI); needs internet access, otherwise system fonts are used |

## npm scripts

| Command | What it does |
|---|---|
| `npm run dev` | Starts Vite on port 3000, reachable on the local network (`--host`) |
| `npm run build` | Builds optimised static files into `dist/` |
| `npm run preview` | Serves the production build on port 4173 for testing |
| `npm run lint` | Type-checks the project with the TypeScript compiler |

## Project structure

```
flipbook-product-catalogue/
  index.html            page shell, fonts, social-preview meta tags
  package.json          dependencies and scripts
  README.md             quick start, customisation and deployment guide
  FIVERR_GIG.md         ready-to-paste Fiverr listing, packages, FAQ
  vite.config.ts        React + Tailwind plugins, "@" path alias
  src/
    main.tsx            entry: applies brand colours, title, meta description
    App.tsx             owns all state; assembles header, stage, controls, modals
    index.css           Tailwind import, fonts, 3D helper classes, scrollbars
    config/
      brand.ts          name, tagline, colours, contact, closing links, credit
      pages.ts          the ordered list of catalogue pages
    components/         21 components (stage, controls, modals, page templates)
    utils/audio.ts      Web Audio page-turn sound
    assets/images/      18 .webp images
```

# 4. Architecture

The application is a single-page React app with one state owner. `App.tsx` holds the current page number and every view toggle, and passes values and callbacks down to presentational components. There is no router, store or network layer.

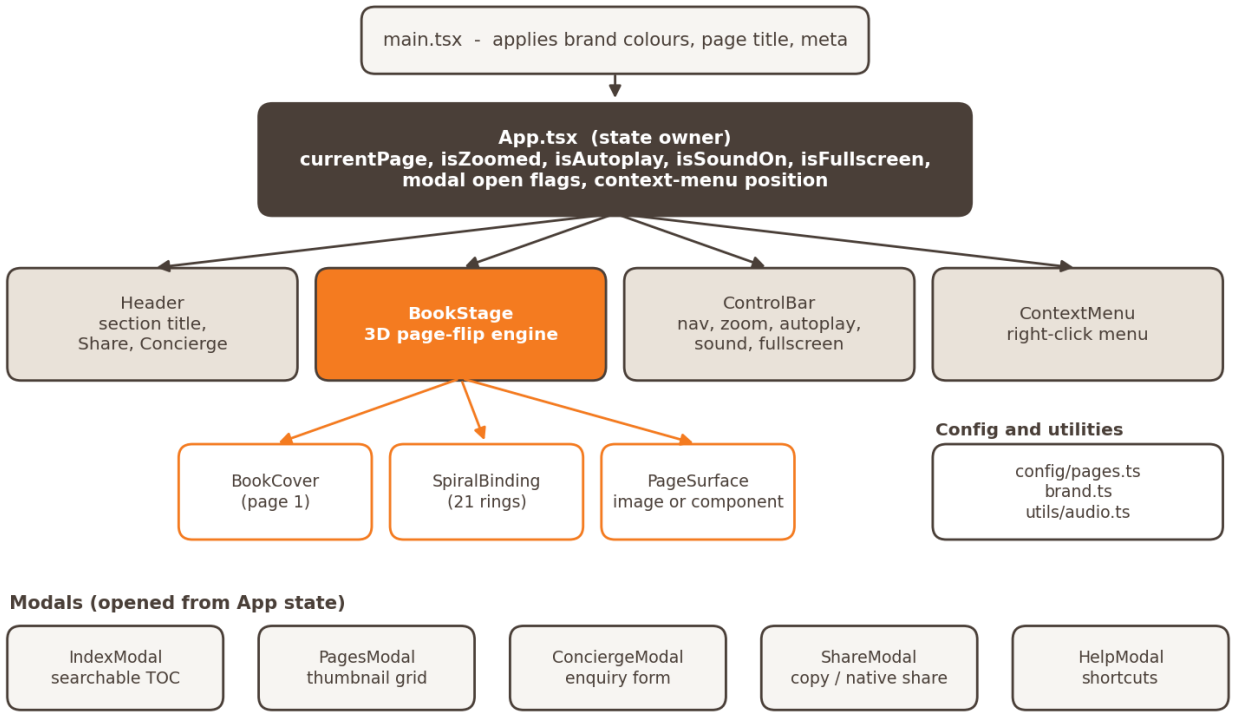



*Figure 1 - Component architecture*

## State held in App.tsx

| State | Purpose |
|---|---|
| currentPage | 1-based page number; clamped between 1 and the total page count |
| isZoomed / isAutoplay / isSoundOn / isFullscreen | View toggles, shared by the control bar and the right-click menu |
| isIndexOpen, isPagesOpen, isConciergeOpen, isShareOpen, isHelpOpen | One flag per modal |
| contextMenu {isOpen, x, y} | Position of the custom right-click menu |

Because every navigation source (buttons, keyboard, thumbnails, index, autoplay, right-click menu) writes to the same `currentPage` state, the page-flip engine reacts to one value only, which keeps behaviour consistent across all of them.

# 5. The Page-Flip Engine

The 3D effect lives in `BookStage.tsx` (about 480 lines) with help from `SpiralBinding.tsx`. Rather than relying on a third-party flipbook library, it implements its own animation loop.

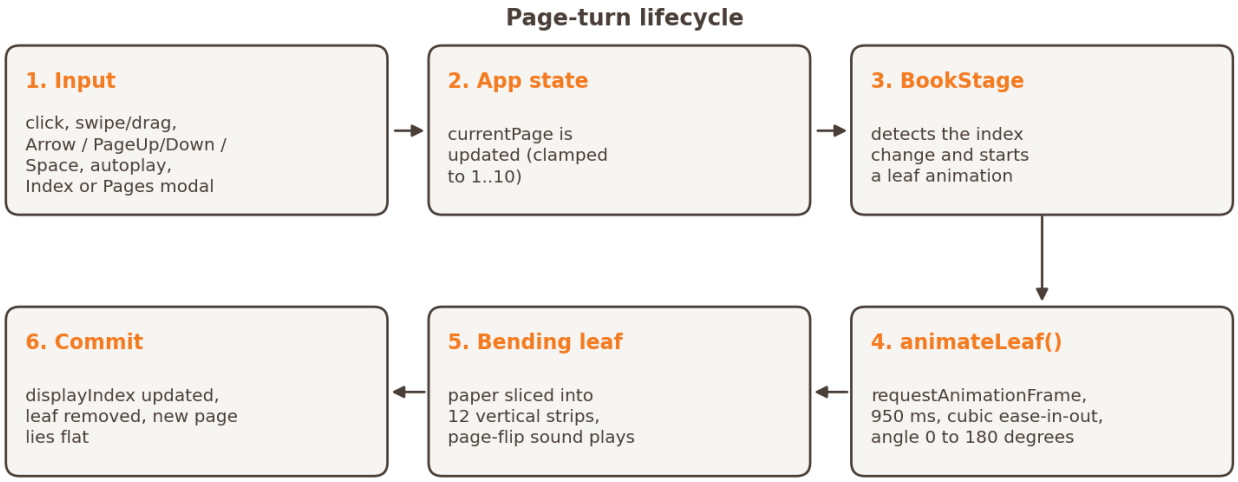



*Figure 2 - What happens when the page changes*

## Key mechanics

| Mechanic | Implementation |
|---|---|
| Bending paper | The turning leaf is sliced into 12 vertical strips (`STRIPS = 12`) so each strip can rotate slightly differently, producing a curved sheet |
| Animation | Custom `requestAnimationFrame` loop; a full turn lasts 950 ms (`TURN_MS`) using a cubic ease-in-out curve |
| Leaf angle | 0 degrees = page flat on the right; 180 degrees = fully turned over the spiral to the left |
| Perspective | CSS perspective of 2600 px on the stage, with preserved 3D transforms and hidden back faces |
| Drag-to-peel | Pointer events with pointer capture; an 8 px dead zone, progress measured over 80% of the page width, commit when progress exceeds 0.28 |
| Performance | Page faces are memoised (`PageSurface`) so the 60 fps animation never re-renders page content |
| Binding | `SpiralBinding` draws 21 rings by default (configurable via a `count` prop) |
| Keyboard | Right Arrow, Page Down or Space = next; Left Arrow or Page Up = previous; ignored while typing in inputs |
| Sound | `utils/audio.ts` synthesises ~0.28 s of band-pass filtered pink noise (sweeping about 1400 to 750 Hz) per turn |

**Design note.** Pages are rendered by `PageSurface`, which picks what to draw in this order: the cover component for page 1, then a custom React component if the page defines one, then a plain image. This is why a page can be either a flat picture or a fully interactive layout.

# 6. The 10-Page Demo Catalogue

The page list is defined in `src/config/pages.ts`. Each entry has an id, page number, header label (`sectionTitle`), index label (`indexTitle`), an image, an optional custom component and a description used as alt text.

| # | Section title | Rendered as | Image shown |
|---|---|---|---|
| 1 | About us and introduction | BookCover component (cover image) | 01_cover-page |
| 2 | The flipbook concept | FlipbookConceptPage component | 02_flipbook-concept-render |
| 3 | Collection directory and index | FurnitureTableOfContentsPage component | 03_collection-directory |
| 4 | Executive seating and ergonomics | Full-page image | 04_page-modern-office-table |
| 5 | Lounge and acoustic pods | Full-page image | 05_page-director-table |
| 6 | Desking and architectural tables | Full-page image | 06_page-conference-table-ravishing |
| 7 | Storage and partitions | Full-page image | 07_page-conference-table |
| 8 | Materials and sustainability | Full-page image | 08_page-curated-bookshelf |
| 9 | Architect and BIM toolkit | Full-page image | 09_page-bookshelf-range |
| 10 | Contact and thank you | BackCoverShowroomsPage component | 10_back-cover |

Page images 4 to 9 are six designed product-page layouts (office table, director table, conference tables and bookshelves). Page-turn images below are exactly the files used by the app.

## Page artwork used in the flipbook




*Page 4 image - "Modern Office Table" layout*




*Page 5 image - "Director Table" layout*




*Page 6 image - "Ravishing Conference Table" layout*

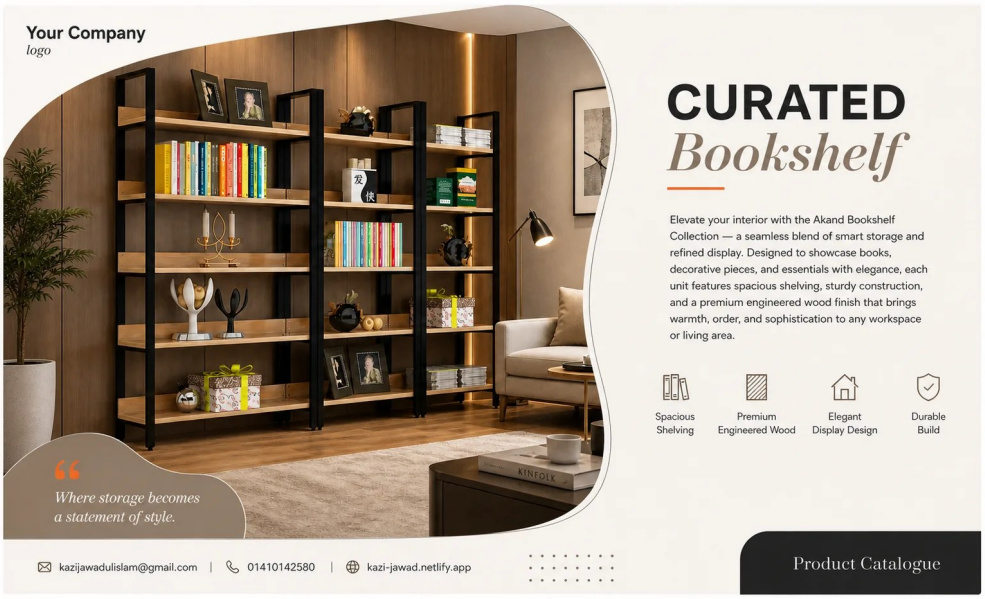



*Page 7 image - "Conference Table" layout*

## Page artwork (continued)



*Page 3 image - collection directory*




*Page 8 image - "Curated Bookshelf" layout*




*Page 9 image - "Bookshelf" range layout*

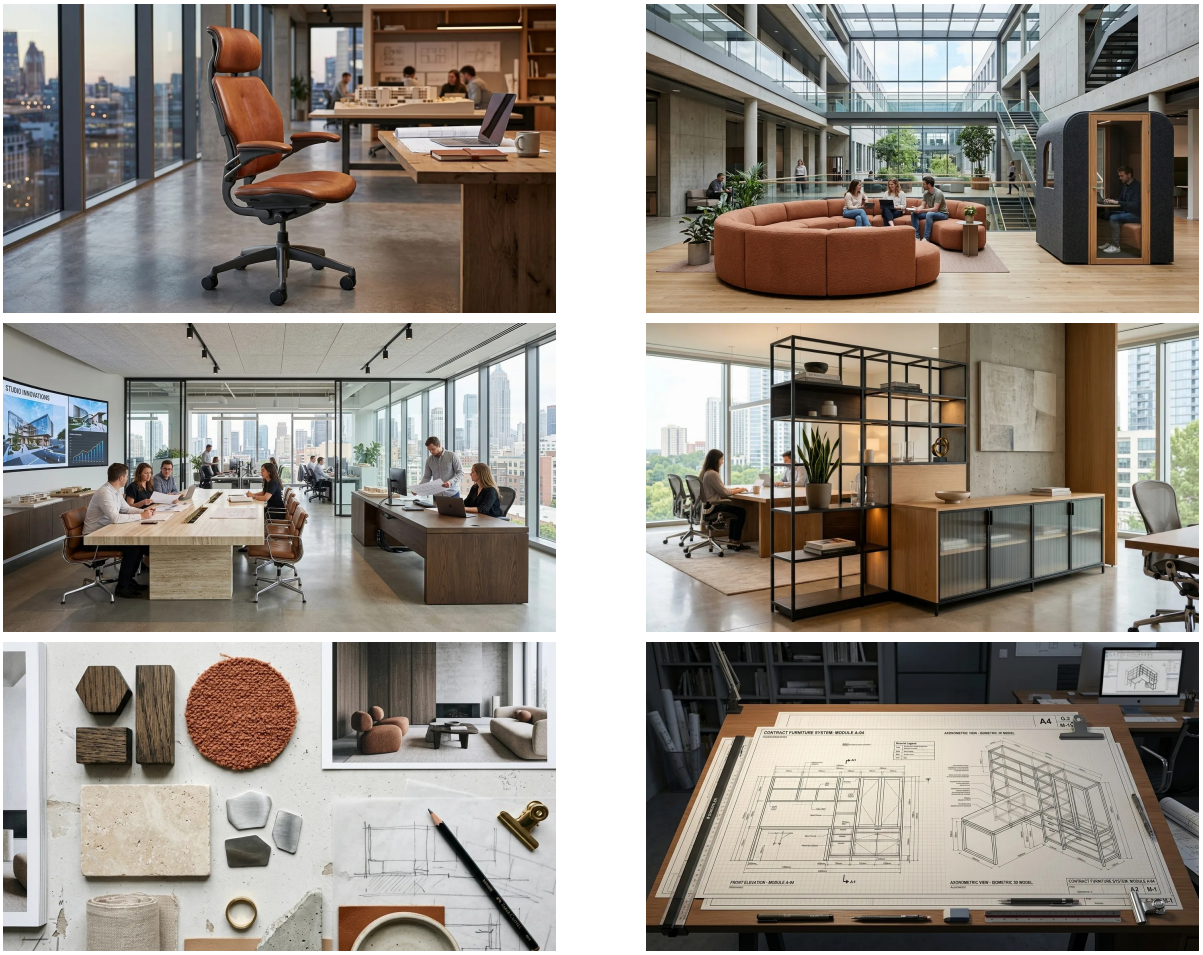



*Page 10 image - back cover*

# 7. Ready-Made Page Templates

Six additional page components are included in `src/components`, each paired with its own image. They show how a page can be a designed React layout instead of a flat picture, and serve as templates (the README points to `ExecutiveSeatingPage.tsx` as the model to copy).

| Component | Image | Theme |
|---|---|---|
| ExecutiveSeatingPage | 11_component-executive-seating | Task and executive chairs, finishes picker |
| LoungeAcousticPage | 12_component-lounge-acoustic | Modular lounge seating and acoustic pods |
| DeskingTablesPage | 13_component-desking-tables | Boardroom tables and height-adjustable desking |
| StoragePartitionsPage | 14_component-storage-partitions | Credenzas and room partitions |
| MaterialsSustainabilityPage | 15_component-materials-swatches | Materials, finishes and sustainability |
| ArchitectBimToolkitPage | 16_component-bim-toolkit | CAD / BIM download resources |



*Images used by the six page templates: executive seating, lounge and acoustic pods, desking and tables, storage and partitions, materials swatches, and the BIM toolkit.*

> **Note.** In the current `pages.ts`, pages 4 to 9 use flat images, so these six components are included in the source but are not wired into the page list. To use one, import it in `pages.ts` and set `component: ExecutiveSeatingPage` on the matching entry.

# 8. Controls and Interface Reference

## Layout

The screen is a three-part column: a **header** (current section title, a "live view" pill, Share and Concierge buttons), the central **book stage**, and a bottom **control bar**. Everything else opens as a modal on top.

## Ways to turn pages

| Method | How |
|---|---|
| Mouse / touch | Click the page edge, or drag the page and release |
| Keyboard - next | Right Arrow, Page Down or Space |
| Keyboard - previous | Left Arrow or Page Up |
| Control bar | First, previous, next, last buttons and a jump-to-page number field |
| Index modal | Click any entry in the searchable table of contents |
| Pages modal | Click any thumbnail |
| Right-click menu | Previous, next, cover page, back cover |
| Autoplay | Automatic every 4.5 s, loops at the end |

## Control bar buttons

Table of contents, all-pages thumbnails, first page, previous, jump-to-page, next, last page, zoom (or reset zoom), start/pause slideshow, mute/enable page-flip audio, enter/exit fullscreen, catalogue shortcuts and help, and "More Options" (mobile overflow).

## Modals

| Modal | Contents |
|---|---|
| Index | Searchable list of pages; highlights the current page |
| Pages | Thumbnail grid of every page; auto-populates from pages.ts |
| Concierge | Contact details (phone, email, address, hours) and an enquiry form with name, email, project scope (Commercial Office Space, Executive Boardroom, Hospitality and Lounge, Bespoke Architectural Project) and message |
| Share | Copy link, native share sheet, email |
| Help | Keyboard and gesture guide (next, previous, 3D flip, zoom, fullscreen) |

# 9. Customisation Guide

## 1. Brand and contact details - src/config/brand.ts

This single file white-labels the catalogue. The accent colour is applied across the whole UI through CSS variables set in `main.tsx`.

| Field | What it controls | Current value in the project |
|---|---|---|
| name, shortName | Brand name in the UI, page title and modals | "Your Company" |
| tagline, subtitle | Marketing line and the "Product Catalogue" label | "Durability. Smart Design. Innovation." / "Product Catalogue" |
| siteDescription | Meta description | Interactive flipbook description |
| colors | Accent, hover, dark UI, surfaces, text | Accent #F47B20, hover #E06D16, dark #4A3F38 |
| contact | Phone, email, address, hours shown in the Concierge modal | Demo phone and address; the email is the developer's own |
| closing | Badge, heading, message and link list for the final page | Email, LinkedIn, GitHub and website links |
| credit | Small "Developed by" footer line; `show: false` hides it | "Developed by Kazi Jawad", shown |

## 2. Pages - src/config/pages.ts

```
{
  id: 4,
  pageNumber: 4,
  sectionTitle: "EXECUTIVE SEATING & ERGONOMICS",  // header label
  indexTitle:   "03. Executive Seating",           // index / thumbnails
  image:        page4Image,                        // full-page image
  description:  "Short description (alt text)",
}
```

- **Image-only page**: import the image and set `image`. Recommended 3:4 portrait, about 1500 x 2000 px, .webp or .jpg under 200 KB.
- **Custom page**: add `component: YourPage`.
- **Add or remove pages** by adding or removing entries; page count, index, thumbnails and navigation update automatically.

## 3. Images and social preview

Drop artwork into `src/assets/images/` and update the imports at the top of `pages.ts`. For link previews, edit the Open Graph and Twitter tags in `index.html`.

# 10. Build and Deployment

## Run locally

```
npm install
npm run dev        # http://localhost:3000
npm run build      # outputs to /dist
npm run preview    # test the production build (port 4173)
```

## Deploy

| Host | How |
|---|---|
| Netlify / Vercel / Cloudflare Pages | Connect the repository; build command `npm run build`; output directory `dist` |
| cPanel / shared hosting | Upload the contents of `dist/` to `public_html` |
| GitHub Pages | Add `base: '/repo-name/'` in `vite.config.ts`, then publish `dist/` |

## Performance (as stated in the README)

The README states the catalogue loads fast, with about 2 MB of images and about 93 kB of gzipped JavaScript. These numbers were not re-measured for this document.

## Commercial packaging (FIVERR_GIG.md)

The project ships with a complete Fiverr gig listing: title, description, tags, packages, extras, FAQ, buyer requirements, a gallery plan, a per-order delivery checklist and message templates. The suggested packages are:

| | Basic | Standard | Premium |
|---|---|---|---|
| Name | Starter Flipbook | Branded Catalogue | Full Custom Catalogue |
| Pages | Up to 10 | Up to 20 | Up to 40 |
| Delivery | 3 days | 5 days | 10 days |
| Revisions | 1 | 3 | Unlimited |
| Deployment to client host | No | Yes | Yes |
| Working enquiry form | No | Yes | Yes |
| Custom page layouts and domain setup | No | No | Yes |
| Suggested price | $50 | $120 | $280 |

Suggested extras range from $15 (PDF to optimised page images) to $40 (second-language version).

# 11. Review Notes and Recommendations

These points come from reading the source. None stops the project working; they are worth checking before delivering it to a client.

| Finding | Detail | Suggested action |
|---|---|---|
| Share link page number is not used | The Share modal builds a link ending in `?page=N`, but no code reads that parameter, so opening a shared link always starts at page 1 | Read `page` from the URL in `App.tsx` to set the initial `currentPage` |
| Enquiry form does not send data | The form shows a confirmation and closes, but submits nowhere (the README says this too) | Connect to Formspree, EmailJS or your own endpoint in `ConciergeModal.tsx` |
| Some demo text is hard-coded | `BookCover.tsx` contains the text "FORMA" and the showroom cities, and the page components contain furniture copy, so changing `brand.ts` alone will not rebrand them | Search the components for demo text before white-labelling |
| Page descriptions do not match images | Descriptions in `pages.ts` for pages 4 to 9 mention seating, pods and BIM, while the images show tables and bookshelves | Update descriptions (they act as alt text) to match the artwork |
| Personal details in brand.ts | The default contact email and the closing-page links are the developer's own, while the company name is a placeholder | Replace with the client's details for every delivery |
| Six page components unused | Executive seating, lounge, desking, storage, materials and BIM components exist but are not referenced by `pages.ts` | Wire them in, or remove them to keep the delivery lean |
| Static page title and preview text | `index.html` uses generic "Product Catalogue" text for the title and social tags (the title is overridden at runtime, social tags are not) | Set Open Graph text per client |
| node_modules shipped in the zip | The uploaded archive includes `node_modules` (about 150 MB unpacked, with a Windows-specific native binary) | Exclude it from deliveries, as the Fiverr checklist already advises |
| Fonts need internet | Google Fonts are loaded from the web; offline visitors see fallback fonts | Self-host fonts if offline use matters |

**Summary.** The project is a clean, dependency-light static React template with a custom 3D page-flip engine, a rich set of viewing controls and a ready-made sales package. With the small fixes above, it is ready to be re-branded and delivered as a catalogue product.

## Appendix - image files used in this document

All 16 artwork images (01 to 16) are the actual image assets referenced by the project source, converted from .webp to .jpg for portability. Images 17 and 18 are diagrams created for this document. All are supplied in the accompanying zip.